

INTEGRANTES DEL EQUIPO:  
SANTIAGO RODRIGUEZ JAYRO ALJENADRO  
AVENDAÑO JIMENEZ ERICK  
CIACA RIVERA ANTONIO

# APACHE KAFKA

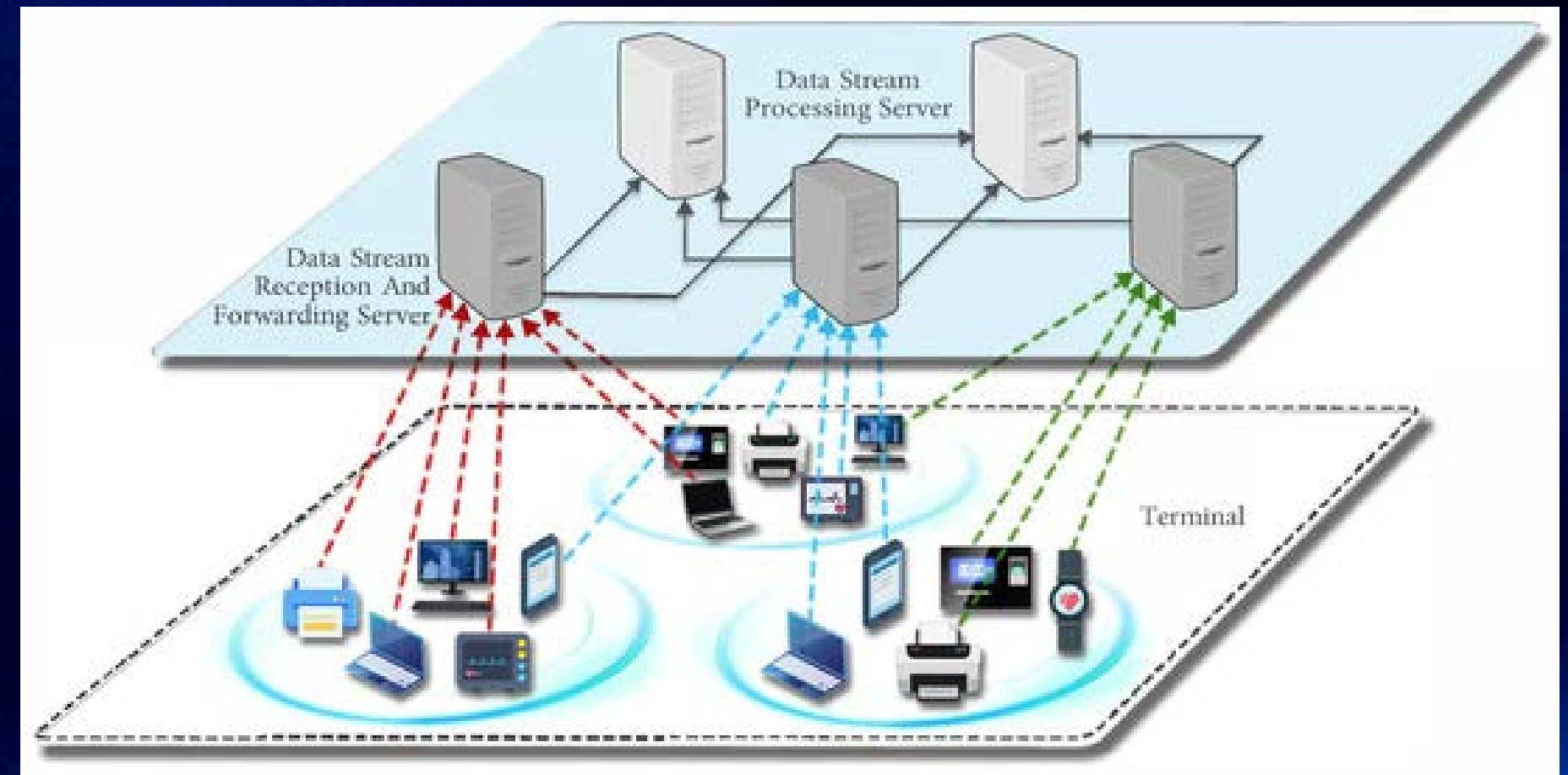
# DEFINICIÓN Y FUNCIÓN PRINCIPAL



Kafka es un sistema de mensajería distribuido basado en eventos que permite:

- Publicar flujos de datos.
- Almacenar registros de eventos.
- Procesar información en tiempo real.
- Integrar aplicaciones y servicios web.
- Analizar datos de manera continua.

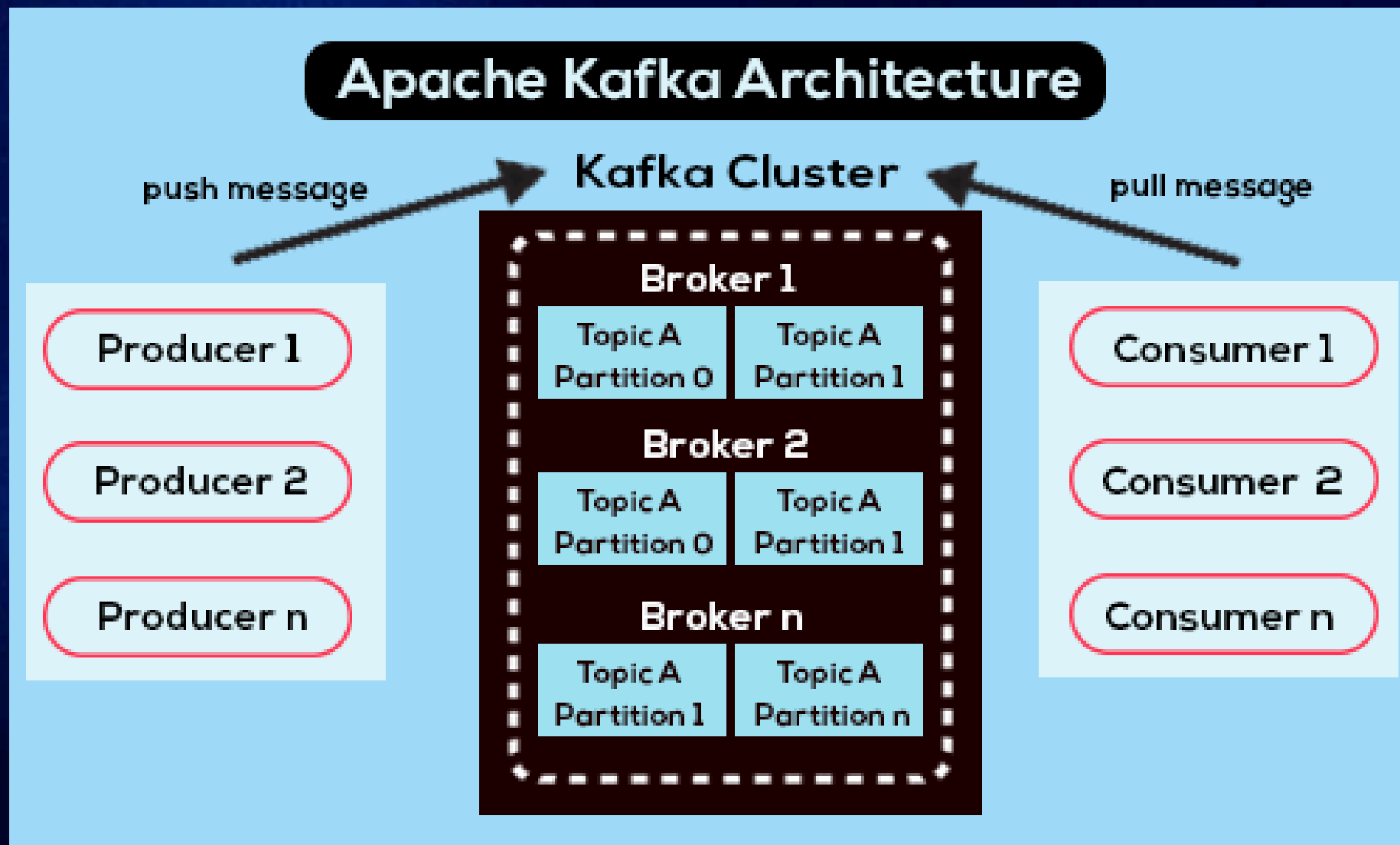
A diferencia de un gestor de colas tradicional, Kafka funciona como un **log de confirmación inmutable** y ordenado, replicado en múltiples servidores.



## FUNCION PRINCIPAL

Es actuar como un intermediario entre aplicaciones, permitiendo que los datos se transmitan de forma eficiente entre productores y consumidores sin que estos dependan directamente entre sí.

# ARQUITECTURA DEL CLÚSTER



## Productores y consumidores

Las aplicaciones web (Producers) envían eventos al broker, mientras que otros servicios (Consumers) los procesan de forma asíncrona.

## Kafka Brokers

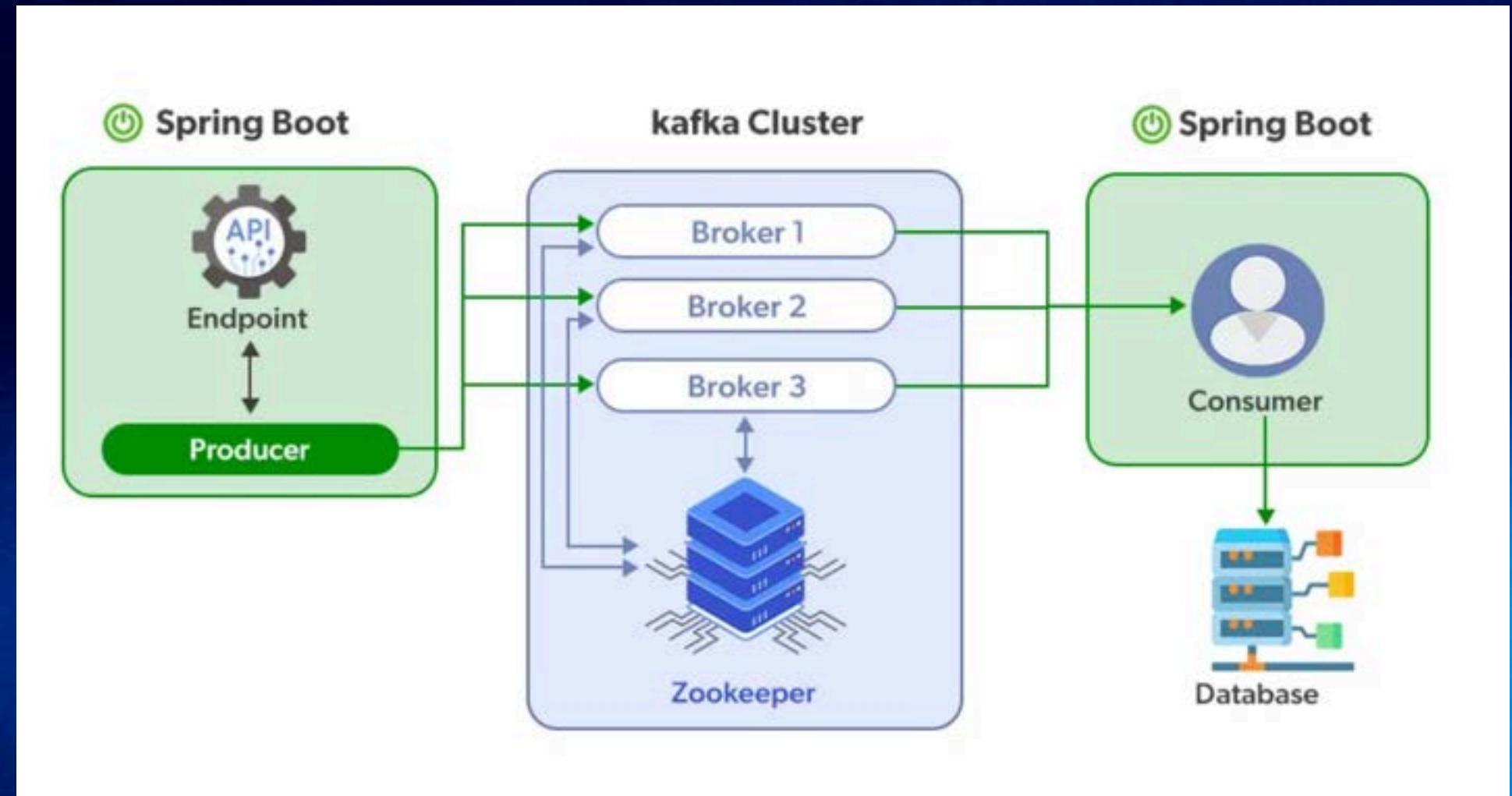
Nodos del servidor que almacenan y sirven datos. La replicación entre brokers garantiza la tolerancia a fallos en la web.

# PRODUCTORES Y CONSUMIDORES

**Productores:** Publican eventos. No necesitan esperar confirmación del consumidor.

**Consumidores:** Leen a su propio ritmo (modelo Pull).  
Kafka guarda su posición usando un Offset.

**Grupos de Consumidores:** Permiten dividir el trabajo. Si un grupo tiene 3 miembros, cada uno procesa 1/3 de las particiones.



# ¿PARA QUÉ SIRVE EN ENTORNOS WEB?

## Analítica Instantánea

Captura el comportamiento del usuario en vivo (clics, búsquedas, carritos) y calcula recomendaciones dinámicas sin impactar la base de datos de producción.



## Microservicios Reactivos

Habilita Arquitecturas Orientadas a Eventos (EDA). Los servicios web se comunican emitiendo eventos de forma asíncrona, eliminando dependencias directas.

## Agregación de Logs

Centraliza trazas de error, métricas de rendimiento y datos de infraestructura de cientos de servidores web en un único pipeline unificado



# ¿CÓMO FUNCIONA EL FLUJO DE DATOS?

## Productores y Consumidores

Los Productores (ej. servidores web Node.js/Python) publican eventos en Kafka sin esperar respuesta. Los Consumidores (ej. microservicios de cobro) extraen datos mediante un modelo "Pull" a su propio ritmo, evitando caídas por saturación.



## Temas, Particiones y Offsets

Los mensajes se agrupan en Temas (Topics). Cada tema se divide en Particiones distribuidas entre servidores para paralelizar lecturas. El Offset es un puntero único secuencial que le dice al consumidor exactamente dónde se quedó leyendo.

# KAFKA FRENTE A COLAS TRADICIONALES

| Característica               | Apache Kafka                                    | Mensajería Tradicional (RabbitMQ, JMS)           |
|------------------------------|-------------------------------------------------|--------------------------------------------------|
| <b>Patrón de Flujo</b>       | Basado en Pull (Consumidor controla su tasa)    | Basado en Push (Broker empuja al consumidor)     |
| <b>Persistencia de Datos</b> | Log inmutable durable (almacenamiento en disco) | Transitoria (los mensajes se borran tras leerse) |
| <b>Rendimiento Crítico</b>   | Masivo (>1 millón de mensajes por segundo)      | Moderado (generalmente miles por segundo)        |
| <b>Consumo Múltiple</b>      | Múltiples consumidores leen el mismo dato       | Un mensaje se entrega a un único consumidor      |

# GUÍA DE INSTALACIÓN (KRAFT)

## 1. Preparación

Requiere **Java 11+**. Descarga el binario oficial (.tgz) y descomprime.

```
tar -xzf kafka.tgz
```

## 2. Configuración

Genera un Cluster ID único para tu infraestructura web.

```
sh storage.sh random-uuid
```

## 3. Inicialización

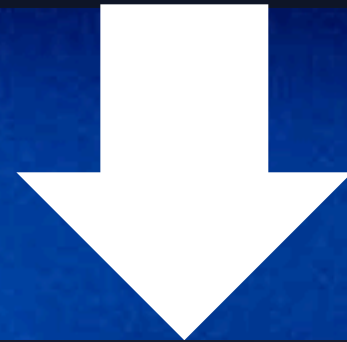
Formatea el log y arranca el broker en el puerto 9092.

```
sh server-start.sh
```

# USO PRÁCTICO: CLI

## Enviar Datos (Producer)

```
kafka-console-producer.sh  
--topic web-events  
--bootstrap-server localhost:9092  
> {"user": "123", "click": "banner"}
```



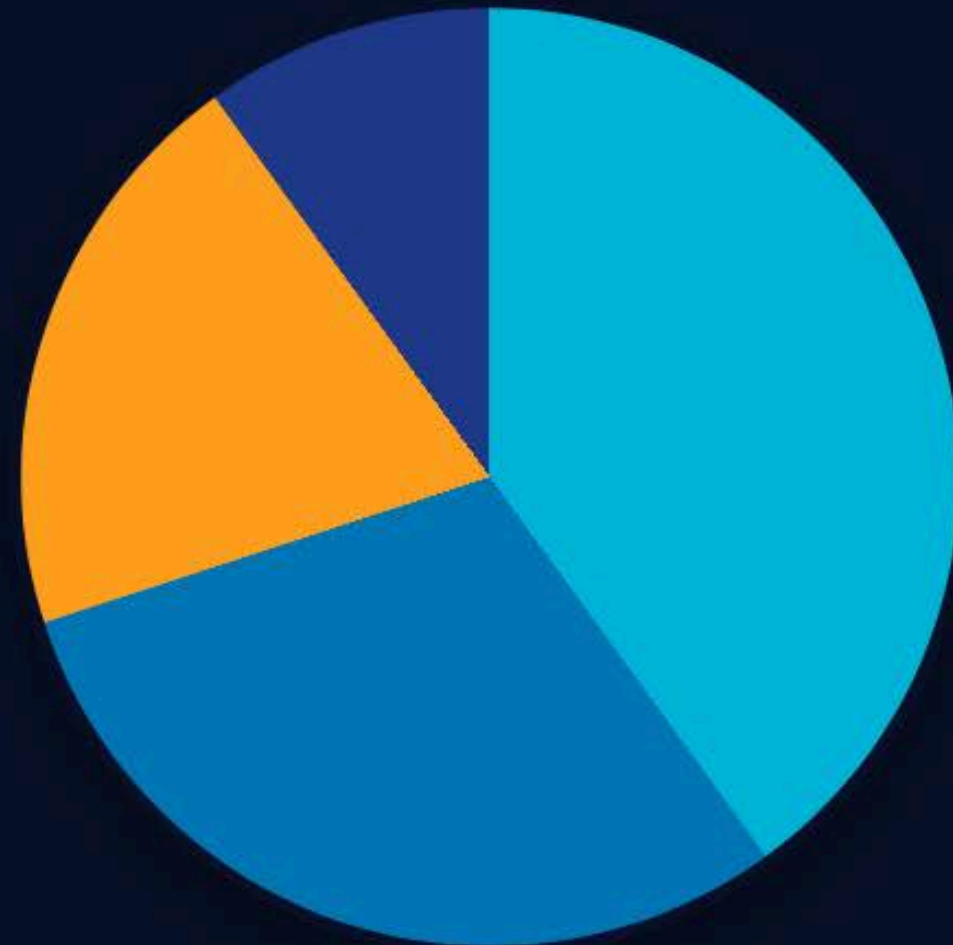
## Leer Datos (Consumer)

```
kafka-console-consumer.sh  
--topic web-events  
--from-beginning  
--bootstrap-server localhost:9092
```

**CLI significa Command Line Interface (en español, Interfaz de Línea de Comandos).**

**Es una herramienta de texto que te permite interactuar con un programa de software escribiendo comandos en una terminal o consola, en lugar de hacer clic en botones o interfaces gráficas (como ventanas o aplicaciones de escritorio).**

# ¿DÓNDE SE APLICA KAFKA HOY?



- 40% Actividad Web (Clics, telemetría de usuario)
- 30% Mensajería e Intercomunicación de Microservicios
- 20% Procesamiento de Métricas y Alertas en Vivo
- 10% Replicación y Pipelines de Datos (ETL)

# ECOSISTEMA EXTENDIDO



## Kafka Connect

Framework para integrar bases de datos (SQL/NoSQL) sin escribir código.



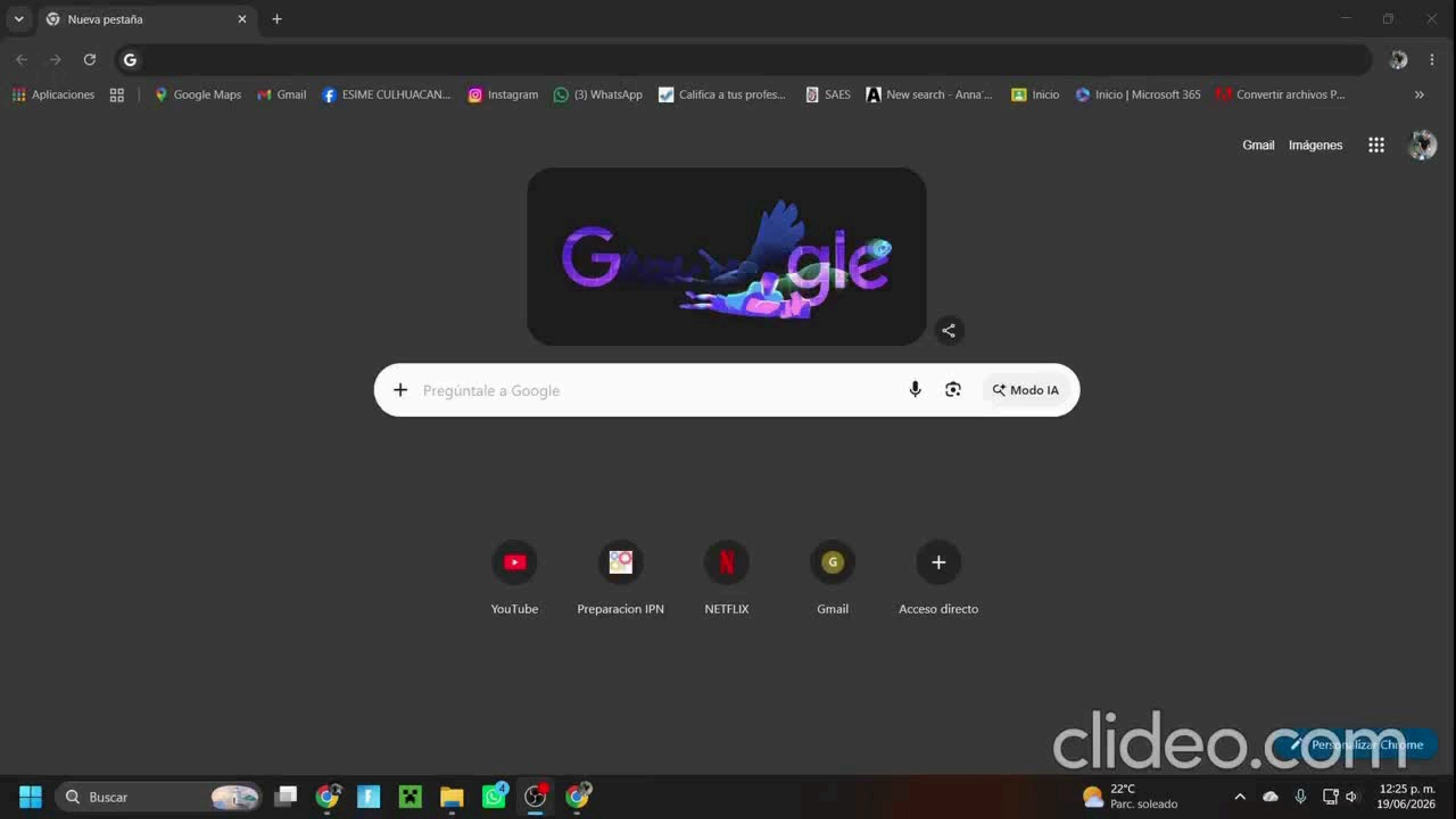
## Kafka Streams

Biblioteca para procesar datos en tiempo real (filtrado, agregaciones).



## KRaft Mode

El nuevo protocolo de consenso que elimina la dependencia de ZooKeeper.

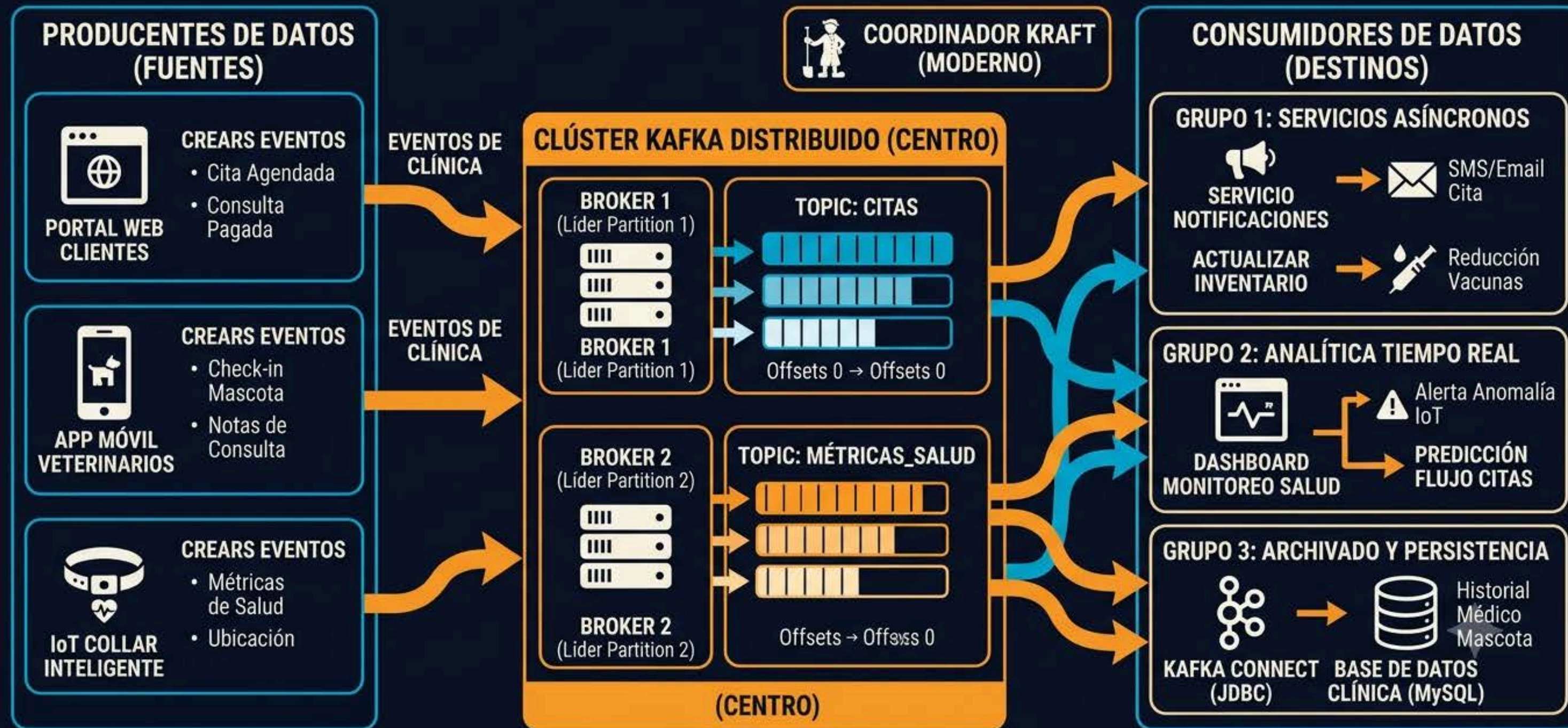


# DE LINKEDIN AL MUNDO

**Originalmente desarrollado en LinkedIn para resolver problemas de escalabilidad en el rastreo de actividad. Fue donado a la Apache Software Foundation en 2011. Hoy es el estándar de facto para el procesamiento de datos en tiempo real en empresas como Netflix, Uber y Spotify.**



# ARQUITECTURA DE KAFKA EN SISTEMA VETERINARIO (ACCESIBLE)



## Productores: ¿Quiénes generan la información en la clínica?

### [PORTAL WEB CLIENTES]



- Registra eventos **en tiempo real** cuando un dueño agenda una cita o realiza el pago de una consulta desde su casa.

### [APP MÓVIL DEL PERSONAL]

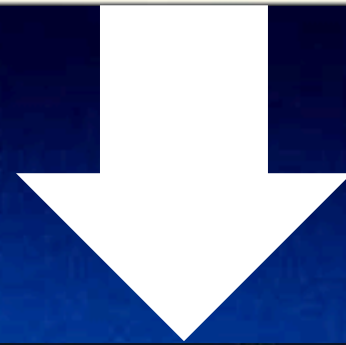


- Permite a los **veterinarios** registrar el **check-in** de una mascota, actualizar el estado de un paciente hospitalizado o añadir notas médicas desde sus tabletas.

### [COLLARES INTELIGENTES IoT]



- **Dispositivos corporales** en mascotas que envían ráfagas constantes de datos biométricos (frecuencia cardíaca y temperatura) sin interrupción.



## Productores: ¿Cómo organiza Kafka los datos de la veterinaria?

### [TOPIC CITAS]



- Canal **inmutable** donde se almacenan cronológicamente todos los eventos de agenda, cancelaciones y facturación.

### [TOPIC METRICAS\_SALUD]

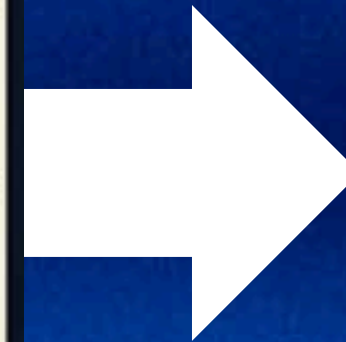


- Canal de **alta velocidad** diseñado para soportar los miles de datos por segundo que envían los collares IoT.

### [ALTA DISPONIBILIDAD]



- Los datos se dividen en **particiones replicadas** entre múltiples servidores (Brokers). Si el Broker 1 falla, el Broker 2 toma el control inmediatamente y la clínica nunca deja de operar.



## Productores: Monitoreo Crítico y Persistencia

### [DASHBOARD DE ALERTAS MÉDICAS]

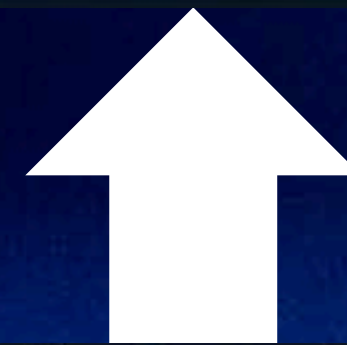


- Un motor analiza el flujo de **metricas\_salud** en tiempo real. Si los signos de un animal hospitalizado son críticos, activa una alerta visual en las pantallas de la clínica.

### [KAFKA CONNECT (PERSISTENCIA)]



- Copia automáticamente todos los eventos del clúster y los guarda de forma estructurada en la Base de Datos central (MySQL, asegurando el historial médico histórico de la clínica).



## Productores: Desacoplamiento: Reacciones en segundo plano

### [SERVICIO DE NOTIFICACIONES]



- Escucha el canal de **citas**. Al detectar una nueva reserva, dispara automáticamente un SMS o correo recordatorio al cliente.

### [SISTEMA DE INVENTARIO]



- Lee las notas médicas en cuanto el veterinario las publica. Si se usó una vacuna o medicamento, reduce el stock de la farmacia de inmediato sin que el veterinario tenga que hacer un registro manual extra.



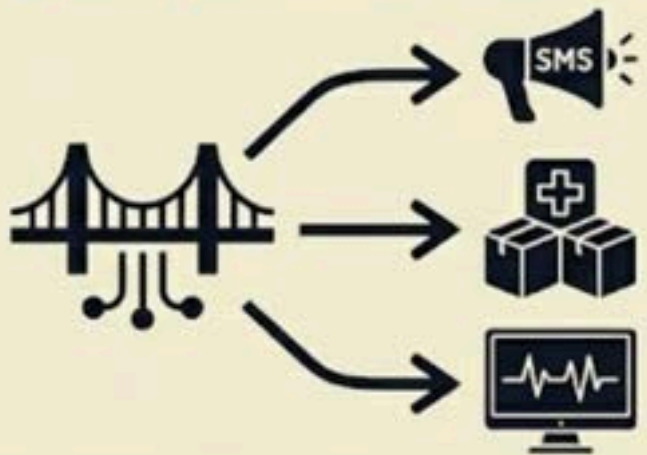
## Productores: ¿Por qué esta arquitectura es superior?

### [ARQUITECTURA SÍNCRONA (TRADICIONAL)]



- Si el sistema de **SMS** fallara, la aplicación del veterinario se **congelaría** esperando una respuesta.

### [ARQUITECTURA ORIENTADA A EVENTOS (KAFKA)]



- La aplicación **solo avisa a Kafka** “Cita registrada” en milisegundos y se **libera**. Cada subsistema (mensajería, inventario, alertas) consume ese mensaje a su **propio ritmo** sin afectar a los demás.



**GRACIAS POR  
SU ATENCION**

